OUTPUT

The following screenshots demonstrate the working output of SoundVault – the Music Library Management System. The application runs locally on Flask's development server at **http://127.0.0.1:5000** and showcases all implemented CRUD features.

1. Add Song

The Add Song panel allows users to enter a song title, artist name, and YouTube URL. Clicking the Add to Library button sends a POST request to the /add_song endpoint and saves the record to the SQLite database.

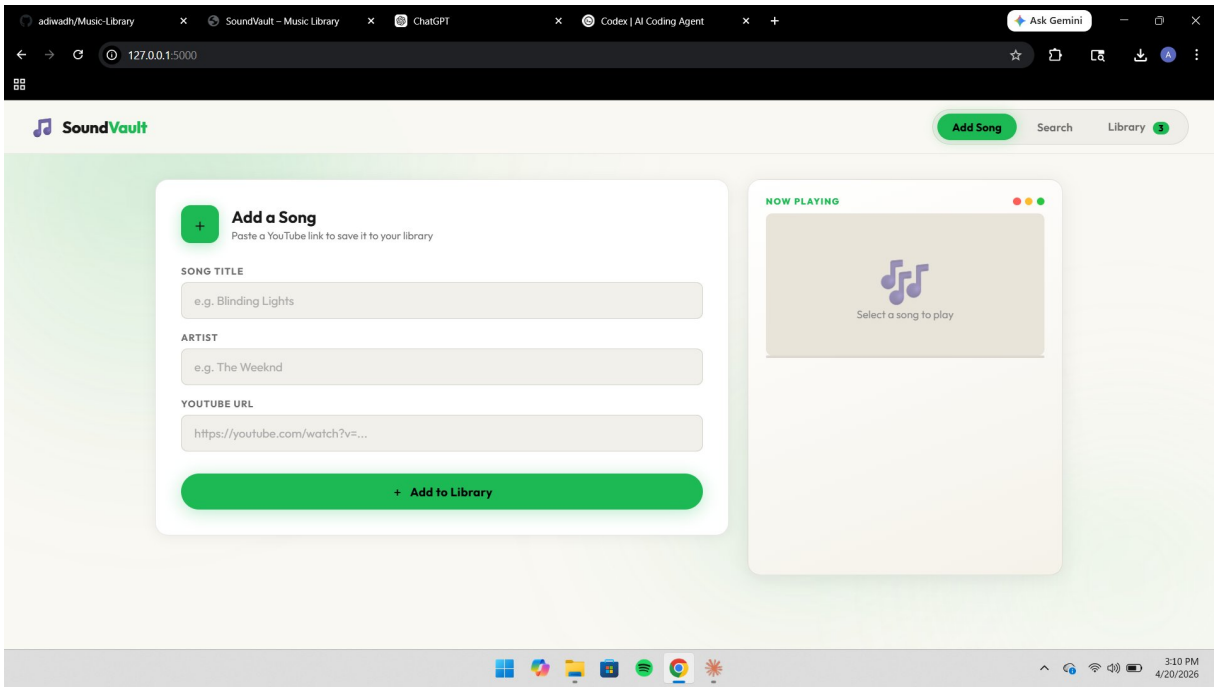

Fig 1: Add Song panel — entering song details with YouTube URL

2. Search Songs

The Search panel enables users to find songs by title or artist using a LIKE query against the database. Results appear dynamically with a play button and favourite heart icon for each matching song.

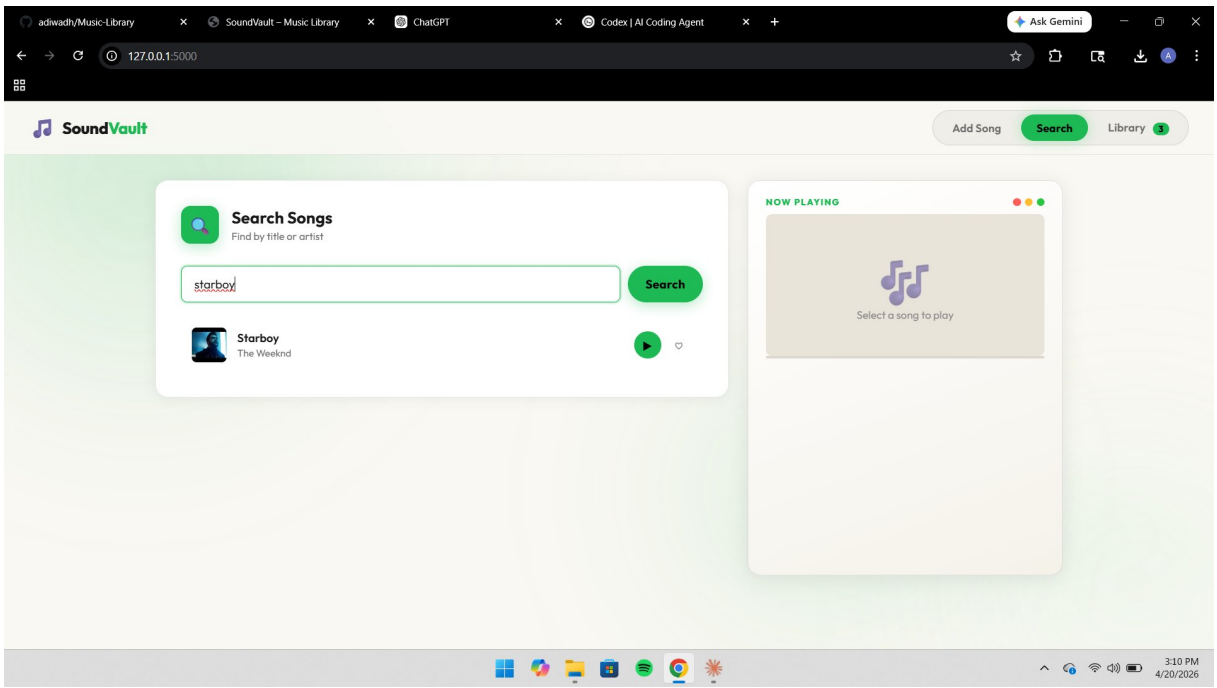

Fig 2: Search results for 'starboy' showing matched song with play controls

3. Your Library – Edit and Delete

The Library panel displays all saved songs ordered by most recently added. Each song card shows a YouTube thumbnail, title, and artist with three action buttons: Play (■), Edit (✏), and Delete (■). The edit and delete functionality represent the Update and Delete operations of the CRUD system.

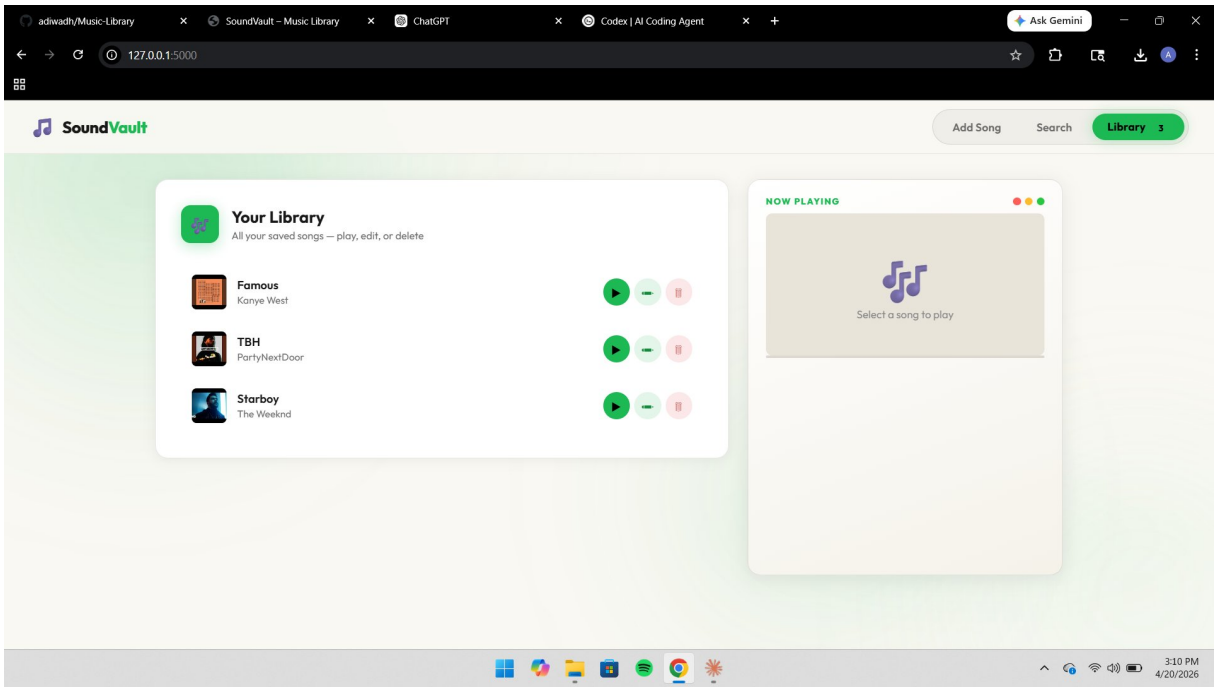

Fig 3: Library view showing 3 songs — Famous, TBH, Starboy — with edit and delete buttons

4. Inline YouTube Playback

Clicking the Play button on any song loads the corresponding YouTube video into the embedded player on the right side of the interface. The Now Playing section displays the song thumbnail, title, artist, and a progress bar, providing a seamless listening experience without leaving the application.

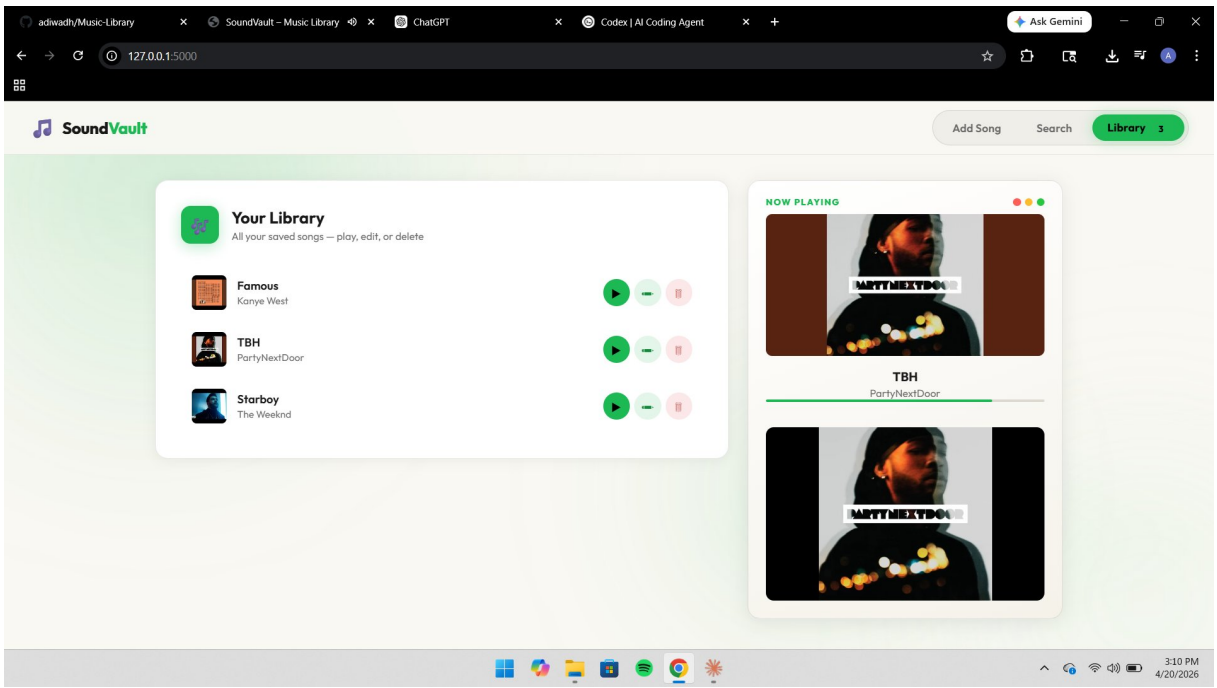

Fig 4: Inline YouTube player showing TBH by PartyNextDoor playing in the Now Playing panel

Feature Summary

Feature	HTTP Method	Endpoint	Status
Add Song	POST	/add_song	✓ Working
View Library	GET	/library	✓ Working
Search Songs	GET	/search?q=	✓ Working
Edit Song	PUT	/update/<id>	✓ Working
Delete Song	DELETE	/delete/<id>	✓ Working
Favourite Song	GET	/favorite/<id>	✓ Working
Inline Playback	—	YouTube iFrame	✓ Working

All features of SoundVault have been successfully implemented and tested. The application demonstrates a complete CRUD cycle backed by a SQLite database and served via a Flask REST API, with a modern, responsive frontend interface.